



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROYECTO DE BASE DE DATOS II

SafeBridge: Orquestador Multi-Motor de RespalDOS y Validación de Integridad

Informe de Factibilidad del Proyecto

FD01

CURSO	Base de Datos II
DOCENTE	Ing. Patrick José Cuadros Quiroga
EMPRESA / EQUIPO	BitCraft Solutions
INTEGRANTE	Sierra Ruiz, Iker Alberto (2023077090)
INTEGRANTE	Cortez Mamani, Julio Samuel (2023077283)

*Sistema *SafeBridge**

Informe de Factibilidad

VERSIÓN 3.0

Control de versiones

Ver-sión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	27/03/2026	Versión Original
2.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	31/05/2026	Actualización Arquitectura a Tauri/Rust
3.0	IASR / JSCM	Ing. P. Cuadros	Ing. P. Cuadros	04/07/2026	Revisión completa con ingeniería inversa del código fuente

Índice General

- 1. Descripción del Proyecto
 - 1.1 Nombre del proyecto
 - 1.2 Duración del proyecto
 - 1.3 Descripción
 - 1.4 Objetivos
- 2. Riesgos
- 3. Análisis de la Situación Actual
 - 3.1 Planteamiento del Problema
 - 3.2 Consideraciones de Hardware y Software
- 4. Estudio de Factibilidad
 - 4.1 Factibilidad Técnica
 - 4.2 Factibilidad Económica
 - 4.3 Factibilidad Operativa
 - 4.4 Factibilidad Legal
 - 4.5 Factibilidad Social y Ambiental
- 5. Análisis Financiero
 - 5.1 Justificación de la Inversión
 - 5.2 Criterios de Inversión (1 Año)
- 6. Conclusiones

1. Descripción del Proyecto

1.1 Nombre del proyecto

SafeBridge: Orquestador Multi-Motor de Respaldos y Validación de Integridad

1.2 Duración del proyecto

Cuatro (4) meses, equivalentes a un ciclo académico universitario (marzo – julio 2026). El análisis de factibilidad económica se proyecta a un horizonte de un (1) año con el fin de evaluar el retorno de la inversión.

1.3 Descripción

En la actualidad, la pérdida de datos representa uno de los principales riesgos operativos para los desarrolladores. Disponer de un archivo de respaldo no garantiza por sí solo que dicho respaldo sea utilizable en una emergencia, ya que los volcados pueden corromperse o estar incompletos.

Ante esta problemática, surge **SafeBridge**, una aplicación de escritorio nativa orientada a desarrolladores individuales y pequeñas empresas para automatizar el ciclo de **Copia de Seguridad (Backup)** y su respectiva **Validación de Integridad**. A diferencia de un simple script de comandos, SafeBridge gestiona conexiones de base de datos de manera segura y centralizada, orquesta las herramientas nativas de volcado (`pg_dump` , `mysqldump` , `sqlcmd` , `mongodump`), calcula hashes SHA-256 para validación de integridad, verifica las firmas de terminación de los archivos (EOF) de manera nativa para comprobar que no hubo corrupción, y almacena un historial inmutable de logs locales de cada operación.

La solución está construida bajo los principios de **Clean Architecture** utilizando tecnologías de última generación: - **(i) Desktop Shell y Backend (Rust/Tauri v2)**: Ofrece un binario ligero, sin depender de un navegador, e interacciona de forma segura con el sistema operativo y el sistema de archivos, con lógica de negocio encapsulada en Rust. - **(ii) Base de Datos Embebida (SQLite)**: Mediante la librería `rusqlite` v0.40, provee una persistencia local rápida y sin configuración para credenciales y logs. - **(iii) Cifrado Local (AES-256-GCM)**: Cifra las contraseñas de las conexiones utilizando `aes-gcm` v0.10 en Rust con nonces aleatorios de 12 bytes. - **(iv) Interfaz de Usuario (React 19 + Tailwind CSS v4)**: Provee una experiencia de usuario moderna, reactiva y con log en tiempo real, conectándose asíncronamente con los comandos de Tauri.

1.4 Objetivos

1.4.1 OBJETIVO GENERAL

Desarrollar una solución de software de escritorio MVP nativa (Windows/Linux) que permita generar volcados (backups) de múltiples motores de base de datos y valide de forma automática la integridad del archivo resultante mediante verificación de firmas y generación de hash SHA-256, proporcionando al usuario una trazabilidad total mediante logs.

1.4.2 OBJETIVOS ESPECÍFICOS

- Diseñar e implementar una arquitectura en **Tauri v2** que desacople la interfaz web (React) de los procesos de sistema nativos ejecutados en Rust.
- Implementar conexiones Multi-Motor (PostgreSQL, MySQL, SQL Server, MongoDB) garantizando que las contraseñas se almacenen cifradas mediante **AES-GCM** en SQLite.
- Integrar la generación de volcados mediante herramientas embebidas (`sidecars` en Tauri como `pg_dump` , `mysqldump` , etc.).
- Verificar nativamente la integridad de los backups leyendo las firmas de terminación (EOF) del archivo y calculando su hash SHA-256.

- Desarrollar un panel (dashboard) analítico para consultar el ratio de éxito/fallo y revisar el historial de logs (tiempos de ejecución y rutas resultantes).
- Proponer una infraestructura en la nube teórica mediante **Terraform**, para futuras versiones, permitiendo el despliegue de almacenamiento S3 y entornos RDS de validación remota para entornos empresariales.

2. Riesgos

A continuación se identifican los principales riesgos que podrían afectar el éxito del proyecto, clasificados por categoría, junto con su probabilidad estimada y la estrategia de mitigación correspondiente.

Categoría	Riesgo	Probabilidad	Estrategia de Mitigación
Técnico	Fallo en la invocación de sidecars (ej. <code>pg_dump</code>)	Media	Validar en Rust la disponibilidad del sidecar antes de la ejecución; encapsular su uso y capturar la salida de error (stderr) para el log.
Técnico	Archivo <code>.bak</code> o <code>.sql</code> generado incompleto	Baja	Verificación nativa (EOF check) implementada en Rust para validar las firmas <code>Dump completed on</code> u análogas según el motor.
Técnico	Pérdida o corrupción de la BD local SQLite	Baja	SQLite es muy confiable, pero se implementará manejo de fallos usando PRAGMA robustos y un archivo único en el perfil del usuario.
Seguridad	Fuga de credenciales almacenadas localmente	Media	Uso estricto de cifrado AES-256-GCM para la columna password en la BD SQLite; las contraseñas no se envían al frontend.
Operativo	Bloqueo de UI durante la generación del backup	Baja	Tauri ejecuta los procesos pesados en <code>async</code> (comandos Tauri) de Rust, permitiendo emitir eventos progresivos y manteniendo la UI en React reactiva.
Humano	Abandono del proyecto por parte de un integrante	Baja	Documentación continua, commits frecuentes en GitHub, uso de GitHub Projects.
Infraestructura	Errores en la definición teórica de recursos Terraform	Media	Usar <code>terraform plan</code> y validaciones estrictas (<code>terraform validate</code>).

3. Análisis de la Situación Actual

3.1 Planteamiento del Problema

Los desarrolladores y administradores de bases de datos de aplicaciones pequeñas suelen generar backups manualmente ejecutando comandos en terminal o mediante clientes gráficos que no validan los resultados. Un simple volcado puede verse interrumpido por pérdida de conexión o disco lleno, generando un archivo corrupto que el usuario asume como válido.

Adicionalmente, manejar conexiones hacia múltiples motores (PostgreSQL para un proyecto, SQL Server para otro) implica usar herramientas dispares, lo que fragmenta el registro de cuándo, dónde y cómo se generaron las copias de seguridad.

SafeBridge MVP aborda el problema centralizando los comandos nativos de respaldo para 4 motores diferentes en una sola UI nativa, evaluando la salud del respaldo creado en disco inmediatamente después de su generación, y brindando un solo lugar donde monitorear el estado de las copias (Dashboard de operaciones).

3.2 Consideraciones de Hardware y Software

El siguiente cuadro resume los recursos tecnológicos evaluados para el desarrollo e implantación del sistema:

Tipo	Componente	Especificaciones	Estado
HARDWARE	Equipos de desarrollo	Laptop/PC procesador x64, 8 GB RAM mínimo, SSD	Disponible
SOFTWARE	Backend y Desktop Shell	Rust 1.75+, Node.js (para compilar React), Tauri v2 CLI	Disponible
SOFTWARE	Frontend	React 19, TypeScript 5.8, TailwindCSS v4	Disponible
SOFTWARE	Base de Datos (Local)	SQLite embebido (via crate <code>rusqlite</code> v0.40)	Disponible
SOFTWARE	Motores de BD de prueba	Imágenes Docker locales (postgres, mysql, etc.) o XAMPP	Disponible
SOFTWARE	Control de versiones	Git + GitHub (repositorio, CI/CD con GitHub Actions)	Disponible
SOFTWARE	IaC (opcional cloud)	Terraform v1.7+ (HashiCorp)	Disponible

Justificación tecnológica: La combinación **Tauri + React + Rust** ofrece un rendimiento inigualable para aplicaciones de escritorio en comparación con Electron o Python. Rust asegura velocidad extrema y garantías de manejo de memoria al manipular archivos grandes o ejecutar procesos nativos de forma paralela y segura (`Command` API / sidecars). React + Tailwind permiten implementar un diseño UI oscuro/técnico muy detallado de forma rápida y moderna. SQLite es el estándar de oro para almacenamiento local estructurado en escritorio.

4. Estudio de Factibilidad

4.1 Factibilidad Técnica

El proyecto es técnicamente viable porque los recursos requeridos se encuentran disponibles:

- **Desktop Framework:** Tauri está estabilizado en su versión 2.0 y su sistema de Plugins (ej. `tauri-plugin-shell`) permite encapsular e invocar los clientes de terminal de las BD (`pg_dump.exe` , `mysqldump.exe` , etc.) sin que el usuario los instale globalmente.
- **Seguridad:** El ecosistema de Rust cuenta con crates auditadas (`aes-gcm` , `sha2` , `rand`) para la criptografía de contraseñas y cálculo de hashes de archivos grandes mediante lectura en chunks asíncronos sin saturar la RAM.
- **Validación Rápida (MVP):** La factibilidad aumenta al basarse en validación EOF en lugar de requerir levantar Docker Desktop en las máquinas de todos los usuarios para cada pequeña validación de archivo.
- **Infraestructura como Código (IaC):** Terraform se define de forma teórica para su uso en la futura validación remota/cloud, asegurando que SafeBridge sea extensible a arquitecturas corporativas.

4.2 Factibilidad Económica

Como aplicación de escritorio, SafeBridge no requiere servidores permanentes, reduciendo el costo a 0 en licenciamiento de base de datos o alojamiento (el usuario almacena el backup en su disco duro local o en un NAS).

4.2.1 COSTOS GENERALES Y DE PERSONAL (DESARROLLO)

Categoría	Descripción	Costo Total Estimado (S/)
Costos Generales	Laptop, disco externo redundante, energía e internet.	~800.00
Costos de Personal	2 desarrolladores junior (S/ 1,200.00 c/u x 4 meses)	~9,600.00
Costos de Licenciamiento	Software Open Source (Tauri, Rust, React, SQLite)	0.00
TOTAL		S/ 10,400.00

4.2.2 ANÁLISIS ECONÓMICO Y COSTOS DE INFRAESTRUCTURA CLOUD (TERRAFORM)

Para cumplir los requisitos técnicos corporativos a futuro y poder sincronizar los archivos generados localmente por el usuario con un entorno seguro en la nube, se ha planificado una infraestructura basada en Amazon Web Services (AWS) gobernada totalmente por **Terraform**.

Recursos Terraform definidos en infraestructura teórica:

```
# main.tf (Ejemplo de backend cloud AWS)
resource "aws_s3_bucket" "safebridge_backups" {
  bucket = "safebridge-secure-backups-prd"
}

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.safebridge_backups.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_db_instance" "validation_rds" {
  allocated_storage = 20
  engine             = "postgres"
  engine_version     = "15.4"
  instance_class     = "db.t4g.micro"
  username           = "admin_validator"
  password           = "..." # administrado por Terraform secrets
  skip_final_snapshot = true
}
```

Estimación de costos cloud (AWS) mensuales si se activara la validación remota:

Recurso Terraform	Tipo AWS	Uso Estimado	Costo (S/) mes
S3 Bucket	Almacenamiento Estándar	50 GB / mes	~ S/ 4.50
RDS PostgreSQL	db.t4g.micro		~ S/ 6.00

Recurso Terraform	Tipo AWS	Uso Estimado	Costo (S/) mes
		40 horas/mes (encendido bajo demanda para verificar)	
Transferencia	Salida a Internet	10 GB	~ S/ 3.50
Total Mensual (Cloud)			~ S/ 14.00

Al gestionar esto con Terraform, la base de datos `validation_rds` puede ser creada y destruida dinámicamente (`terraform apply` / `terraform destroy`), lo que minimiza drásticamente el costo frente a tener servidores activos 24/7.

4.3 Factibilidad Operativa

El sistema es operativamente viable dado que resuelve directamente un proceso engorroso para el desarrollador individual: - **Agrupar Múltiples Motores:** Elimina la necesidad de memorizar las banderas y sintaxis de `pg_dump`, `mysqldump`, `sqlcmd`. - **Registro Centralizado:** En lugar de dejar volcados sueltos, la BD SQLite embebida recuerda qué base de datos generó el `.bak`, con qué nombre, qué día, cuánto duró, su hash SHA-256, y si su firma era válida. - **Desempeño:** Al estar escrita en Rust, la aplicación ocupa mínimos MB de memoria RAM y no interfiere con el trabajo diario del desarrollador.

4.4 Factibilidad Legal

- **Protección de datos personales:** La aplicación opera 100% de manera local. Los archivos de bases de datos de clientes generados mediante backup no salen de la máquina del usuario (salvo que él opte por subirlos a la nube).
- **Licenciamiento Open Source:** React (MIT), Rust (MIT/Apache), SQLite (Dominio Público). La aplicación puede distribuirse gratuitamente o bajo licenciamiento comercial sin restricciones por sus componentes fundamentales.

4.5 Factibilidad Social y Ambiental

- **Impacto Social:** Aumenta la madurez de ingeniería (DevOps) de desarrolladores individuales al introducirlos al ciclo BCDR (Continuidad de Negocio).
- **Impacto Ambiental:** Nulo para la aplicación en sí. Si se usa Terraform con AWS, la política "bajo demanda" (destruir tras usar) en lugar de "always-on" reduce el footprint de carbono en los centros de datos.

5. Análisis Financiero

El análisis financiero asume la perspectiva de la pequeña empresa/estudio independiente. Se evaluarán los beneficios de productividad generados por el uso de SafeBridge.

5.1 Justificación de la Inversión

Beneficios Cuantificables: - **Ahorro de Tiempo:** Reducción del tiempo de generación manual y organización de volcados en al menos 2 horas-hombre por semana (2 horas × S/ 25.00/hora × 52 semanas = **S/ 2,600.00** al año). - **Reducción de Riesgo (Prevención):** Si asumimos 1 incidente de pérdida crítica al año por un archivo corrupto que hubiese sido detectado por SafeBridge, y un costo operativo/daño de negocio valuado conservadoramente en S/ 10,000 con un 50% de probabilidad de ocurrencia sin la herramienta, el beneficio por riesgo mitigado anual es **S/ 5,000.00**.

Beneficio total esperado: **S/ 7,600.00 por desarrollador al año**. Si consideramos una firma de desarrollo con 3 ingenieros, el beneficio sube a **S/ 22,800.00** anuales.

5.2 Criterios de Inversión (1 Año)

Considerando una firma de desarrollo de 3 ingenieros, los indicadores bajo una tasa COK del 10% son:

Indicador	Año 0 (Inversión Inicial)	Año 1 (Beneficio Neto Anual)	Resultado
Inversión total	S/ -10,400.00	—	Costo de desarrollo.
Flujo Caja Año 1	—	S/ 22,800.00	Ahorros y prevención.
VAN (tasa 10 %)	—	S/ 10,327.27	Positivo — Aceptar
B/C	—	2.19	> 1 — Aceptar
TIR	—	119 %	> COK (10 %) — Aceptar

El análisis demuestra que desarrollar SafeBridge generará un retorno sobre la inversión (ROI) significativamente favorable en el primer año debido a la contención del riesgo crítico de pérdida de datos.

6. Conclusiones

- **Factibilidad Técnica — VIABLE:** La arquitectura Rust + Tauri + React es robusta, probada, moderna y sumamente eficiente para invocar sidecars nativos de bases de datos.
- **Factibilidad Económica — VIABLE:** El desarrollo solo consume horas-hombre al estar basado en librerías Open Source. El análisis B/C arroja un ratio superior a 2, validando el retorno de la inversión para firmas de desarrollo. La infraestructura propuesta por Terraform es extremadamente barata.
- **Factibilidad Operativa — VIABLE:** La UI es intuitiva y soluciona un problema directo (la validación real de un backup generado localmente y su almacenamiento en SQLite para consultas futuras).
- **Factibilidad Legal/Ambiental — VIABLE:** Uso exclusivo de licencias permisivas (MIT, Apache) y nulo procesamiento de información personal hacia el exterior.

En conclusión, el proyecto "**SafeBridge: Orquestador Multi-Motor de Respalos y Validación de Integridad**" es un software factible y de alta calidad técnica que puede ejecutarse y completarse exitosamente bajo el stack moderno planteado.